



**OSTİM TECHNICAL UNIVERSITY
FACULTY OF ENGINEERING
CAPSTONE PROJECT REPORT**

**ASTROBOT — AN INTELLIGENT CONVERSATIONAL AI
ASSISTANT WITH MULTIMODAL CAPABILITIES, RETRIEVAL-
AUGMENTED GENERATION AND WEB-BASED WORKFLOW
INTEGRATION**

Project Team

TIDIANE KONATE (220201838)
SIDI MOHAMED SALL (210208992)
AHMED ESSALEM (220201992)
SAAD IBRAHIM HOUSSEIN (210201983)

Project Advisor

Assist. Prof. Dr. Yücel TEKİN

Undergraduate Project

Department of Computer Engineering
MFBP 402 — Graduation Project II

May 2026

APPROVAL

This undergraduate capstone project, entitled “AstroBot — An Intelligent Conversational AI Assistant with Multimodal Capabilities, Retrieval-Augmented Generation and Web-Based Workflow Integration”, prepared by Tidiane Konate, Sidi Mohamed Sall, Ahmed Essalem and Saad Ibrahim Houssein under the supervision of Assist. Prof. Dr. Yücel TEKİN, has been examined by the committee and accepted as an undergraduate capstone project in terms of scope and quality.

Assist. Prof. Dr. Yücel TEKİN

Supervisor

Jury Member Name Surname

Member

Jury Member Name Surname

Member

Head of Department Name Surname

Head of Department

ACKNOWLEDGEMENTS

This graduation project was carried out by TIDIANE KONATE, SIDI MOHAMED SALL, AHMED ESSALEM and SAAD IBRAHIM HOUSSEIN, students from the Faculty of Engineering, Department of Computer Engineering at OSTIM Technical University. We would like to extend our heartfelt gratitude to our esteemed supervisor, Assist. Prof. Dr. Yücel TEKİN, for dedicating her valuable time to us, providing us with his vast knowledge and experience, and guiding us throughout the development of this project.

Our sincere thanks also go to the Rectorate of OSTIM Technical University, the Dean of the Faculty of Engineering, and the Head of the Department of Computer Engineering for providing us with the necessary resources, infrastructure and academic environment that made this work possible. We are equally grateful to all the professors of our department whose lectures and feedback have shaped us into engineers ready to tackle real-world software systems.

We would also like to express our deep appreciation to our families, whose continuous emotional and material support throughout this journey has been the foundation of our perseverance. Without their unwavering encouragement, this accomplishment would not have been possible.

Finally, we thank the open-source community — the maintainers of Node.js, Express, PostgreSQL, n8n, Hugging Face, Mistral AI, KaTeX, highlight.js, Mermaid, Tesseract.js, Cropper.js, Chart.js, pdfkit and pdf-parse — without whose work AstroBot could not exist.

TIDIANE KONATE
SIDI MOHAMED SALL
AHMED ESSALEM
SAAD IBRAHIM HOUSSEIN

ÖZET

Bu çalışma, AstroBot adlı çok modlu yeteneklere sahip akıllı bir konuşma tabanlı yapay zekâ asistanının tasarımını ve gerçekleştirmesini sunmaktadır. Proje, günümüz yapay zekâ araçlarının dağınıklığı sorununu ele alır: sohbet, görsel üretimi, belge arama, OCR ve ses dökümü genellikle ayrı uygulamalarda yer alır ve kullanıcılar en güncel modellere erişmek için birden çok ücretli aboneliğe ihtiyaç duyar. Önerilen sistem; herkese açık bir sohbet arayüzü, bir yönetici paneli ve her kullanıcı mesajını Mistral büyük dil modeline yönlendiren n8n tabanlı bir iş akışı düzenleyicisinden oluşur. Sistem; metin tabanlı sohbet, görsel üretimi, görselden görsele düzenleme, OCR ve görüntü altyazılama, ses dökümü, belgeler üzerinde soru-cevap (RAG), PDF rapor üretimi ve tekrar kullanılabilir iş akışı zincirleri gibi temel işlevleri içerir. Geliştirme süreci yinelemeli sprint'ler hâlinde yürütülmüş, gereksinimler son kullanıcı ve yönetici paydaşlarından elde edilmiştir. Çözüm Docker ile konteynerleştirilmiş, gerçek bir sunucu üzerinde dağıtılmış ve 66 entegrasyon testinden oluşan bir paketle doğrulanmıştır.

Anahtar Kelimeler: Konuşma tabanlı yapay zekâ, büyük dil modeli, RAG, çok modluluk, iş akışı düzenleme, MVP

ABSTRACT

This report presents the design and implementation of AstroBot, an intelligent conversational artificial-intelligence assistant with multimodal capabilities. The project targets the fragmentation of today's AI tooling: chat, image generation, document search, OCR and audio transcription typically live in separate applications, and users juggle several paid subscriptions to reach state-of-the-art models. The proposed system consists of a public chat interface, an administrative dashboard, and an n8n-based workflow orchestrator that routes every user message through a Mistral large-language model. The Minimum Viable Product delivers core functionality including text chat with conversation memory and personas, image generation and image-to-image editing, OCR and visual captioning, audio transcription, retrieval-augmented question answering over uploaded PDF documents, on-demand PDF report generation, and reusable workflow chains, alongside a separate admin service for statistics and user management. Development followed an iterative, Scrum-style process across four sprints, with requirements elicited from end-user and administrator stakeholders. The solution is containerised with Docker, deployed on a live virtual server, and validated by a 66-case integration test suite. Evaluation indicates that the system meets its acceptance criteria and fulfils the goal of bringing several AI modalities together behind a single, free-to-use web application.

Keywords: Conversational AI, large language model, retrieval-augmented generation, multimodal, workflow orchestration, MVP

TABLE OF CONTENTS

	Page
TABLE OF CONTENTS	iv
LIST OF TABLES	vi
LIST OF FIGURES.....	vii
LIST OF SYMBOLS AND ABBREVIATIONS.....	viii
1. Introduction.....	1
1.1. Problem Statement	1
1.2. Project Objectives	1
1.3. Project Scope	2
1.4. Report Organization.....	2
2. General Concepts	3
2.1. Domain Definition	3
2.2. Technologies Used.....	3
2.3. Fundamental Concepts.....	3
3. Related Work.....	5
3.1. Existing Systems.....	5
3.2. Academic Studies.....	5
3.3. Comparative Analysis.....	5
4. Software Process Model.....	7
4.1. Selected Software Development Model.....	7
4.2. Definition of Scrum Roles	7
4.3. Scrum Process.....	7
4.4. Scrum Implementation in the Project.....	8
5. Requirements Analysis	9
5.1. Stakeholders.....	9
5.2. User Requirements.....	9
5.2.1. Functional Requirements	9
5.2.2. Non-Functional Requirements.....	9
5.3. System Specifications	10
6. System Modeling	12
6.1. Context Diagram.....	12
6.2. Use Case Diagram.....	12
6.3. Use Case Descriptions	13
6.4. Sequence Diagrams.....	14
7. System Architecture and Design.....	16

7.1. System Architecture	16
7.2. Structural Models	17
7.2.1. Class / Entity-Relationship Diagram	18
7.2.2. Inheritance (Generalization)	18
7.2.3. Aggregation / Composition	18
7.3. Behavioral Models	18
7.3.1. Event-Driven Model	18
7.3.2. State Diagram	19
8. System Implementation	20
8.1. Development Environment	20
8.2. System Components	20
8.3. Code Structure	21
9. MVP Development	23
9.1. MVP Definition	23
9.2. Implemented Features	23
9.3. System Interfaces	24
9.4. System Demonstration	27
10. Testing and Evaluation	31
10.1. Testing Strategy	31
10.2. Test Scenarios	31
10.3. Test Results	31
11. Results and Discussion	33
12. Conclusion and Future Work	34
12.1. Conclusion	34
12.2. Future Work	34
REFERENCES	35
CURRICULUM VITAE	36
13. Appendices	43
13.1. Appendix A: Source Code	43
13.2. Appendix B: Sprint Backlog	45
13.3. Appendix C: User Guide	46

LIST OF TABLES

	Page
Table 4.1: Scrum roles and the team members who held them.....	7
Table 4.2: Sprint plan and delivered increments.....	8
Table 5.1: Functional requirements.....	9
Table 5.2: Non-functional requirements	9
Table 5.3: Selected REST endpoints and their responsibilities.....	10
Table 7.1: Main database tables and their purpose	17
Table 8.1: Third-party services used by AstroBot	20
Table 10.1: Integration test suite — results by category	31

LIST OF FIGURES

	Page
Figure 6.1: Context diagram of the AstroBot platform.....	12
Figure 6.2: Use-case diagram (end user and administrator).....	13
Figure 6.3: Sequence diagram for a chat message round-trip	14
Figure 7.1: System architecture (client / application / orchestration / data tiers).....	17
Figure 7.2: Entity-relationship model of the PostgreSQL schema.....	17
Figure 7.3: State diagram of a conversation message	19
Figure 8.1: n8n workflow that routes every message through Mistral.....	21
Figure 9.1: Public landing page with the central chat bar	24
Figure 9.2: Features section of the public application.....	24
Figure 9.3: About page with the four-developer team	25
Figure 9.4: Authenticated chat interface (welcome screen)	25
Figure 9.5: Persona selector — seven selectable assistant personas	25
Figure 9.6: Profile settings with an in-browser image cropper	26
Figure 9.7: Admin dashboard — KPIs and activity charts.....	26
Figure 9.8: Admin user-management table	27
Figure 9.9: Admin self-service settings (profile and password).....	27
Figure 9.10: Markdown-rich reply rendered inside a bot bubble.....	28
Figure 9.11: AI image generation example	28
Figure 9.12: On-demand PDF report shown as a download card.....	28
Figure 9.13: OCR analysis of an attached image	29
Figure 9.14: Document Q&A — uploading and binding a PDF (notebook / RAG).....	29
Figure 9.15: Composing a reusable multi-step workflow chain.....	30
Figure 13.1: Sprint timeline of the project	46

LIST OF SYMBOLS AND ABBREVIATIONS

AI	: Artificial Intelligence
API	: Application Programming Interface
BLIP	: Bootstrapping Language-Image Pre-training (image captioning model)
CRUD	: Create, Read, Update, Delete
CSS	: Cascading Style Sheets
DB	: Database
ETA	: Estimated Time of Arrival (used generically)
FR	: Functional Requirement
HTML	: HyperText Markup Language
HTTP	: HyperText Transfer Protocol
JSON	: JavaScript Object Notation
JWT	: JSON Web Token
KaTeX	: LaTeX math typesetting library for the web
LLM	: Large Language Model
MVP	: Minimum Viable Product
NFR	: Non-Functional Requirement
OCR	: Optical Character Recognition
ORM	: Object-Relational Mapping
RAG	: Retrieval-Augmented Generation
REST	: Representational State Transfer
SQL	: Structured Query Language
STT	: Speech-to-Text
TTS	: Text-to-Speech
UI / UX	: User Interface / User Experience
UML	: Unified Modeling Language
VM	: Virtual Machine
VPS	: Virtual Private Server

1. Introduction

1.1. Problem Statement

Modern artificial-intelligence assistants are powerful but fragmented. A user who wants to chat with a model, generate an image, transcribe an audio recording, run optical character recognition on a screenshot, and ask questions over a PDF typically needs four or five different applications, each with its own account, interface, and — increasingly — its own paid subscription. State-of-the-art models sit behind paywalls that put students and small teams at a disadvantage, while self-hosting open models remains complex because every model has its own integration story. Conversations are also treated as disposable: most tools offer flat message lists with no folders, no tags, no way to pin a document as durable context, and no exportable record of what was discussed. Reusable multi-step prompt pipelines are either rebuilt from scratch every time or hidden inside closed agents. There is no single, affordable place that brings conversational AI, multimodal input, document understanding, and reusable workflows together.

1.2. Project Objectives

The objective of this project is to design, implement, and deploy a practical Minimum Viable Product that unifies these capabilities behind one free-to-use web application. Specific objectives are:

- Provide a conversational chat surface backed by a large-language model, with per-session memory, multiple personas, and rich rendering of Markdown, mathematics, code, and diagrams.
- Support multimodal input and output — image generation and image-to-image editing, OCR and visual captioning, and audio transcription.
- Offer document understanding through retrieval-augmented generation over uploaded PDF files, plus a notebook mode that pins a document as durable context.
- Make conversations persistent and exportable, with folders/tags, search, and PDF report generation.
- Provide an administrative dashboard for statistics, user management, and conversation auditing, isolated from the public application.

- Keep the system free to operate by combining free-tier APIs (Mistral, HuggingFace, Groq) and a self-hostable workflow engine, and ship it as a reproducible Docker deployment.

1.3. Project Scope

The project covers the end-to-end software-engineering pipeline for the MVP: requirements analysis, system modelling, architecture and design, implementation, testing, and deployment on a live server. In scope are the public chat application, the administrative dashboard, the n8n orchestration workflow, and the twelve-plus AI-powered features described in this report. Out of scope — and deferred to future work — are native mobile applications, a fine-tuned in-house model, billing and quota enforcement, and large-scale load testing. The system targets a single deployment environment rather than a multi-tenant SaaS offering.

1.4. Report Organization

Section 2 introduces the domain and the enabling technologies. Section 3 reviews existing systems and academic work and positions AstroBot among them. Section 4 describes the software process model. Section 5 presents the requirements analysis, and Section 6 the system models (context, use-case, and sequence diagrams). Section 7 details the architecture and the structural and behavioural design. Section 8 covers the implementation; Section 9 the MVP, its features, interfaces, and a demonstration. Section 10 reports the testing and evaluation, Section 11 discusses the results, and Section 12 concludes with future work. The curriculum vitae of the team members and the appendices (source-code excerpts, sprint backlog, and a user guide) follow the references.

2. General Concepts

2.1. Domain Definition

AstroBot belongs to the domain of conversational AI applications: software systems that expose a large-language model through a chat interface and orchestrate additional tools — image models, speech models, search, and retrieval — to answer user requests. The domain combines natural-language interaction design, web application engineering, and the operational concerns of calling external model APIs reliably and affordably. Within this domain the project sits at the intersection of an end-user productivity tool and a small operational platform with its own administration side.

2.2. Technologies Used

The solution stack is deliberately built from well-supported, mostly free building blocks:

- Frontend — vanilla HTML, CSS and JavaScript (no framework), with KaTeX for mathematics, highlight.js for code, Mermaid for diagrams, Cropper.js for avatar cropping, and Chart.js in the admin dashboard.
- Backend — Node.js 18 with Express 4, JSON Web Tokens for authentication, bcrypt for password hashing, Helmet for HTTP security headers, and express-rate-limit. Document and PDF handling use pdf-parse and PDFKit; OCR uses Tesseract.js.
- AI and models — Mistral medium-latest as the chat model (reached through the n8n AI Agent), HuggingFace Inference for FLUX.1-schnell (image generation), BLIP (image captioning) and MiniLM-L6-v2 (sentence embeddings), Groq Whisper-large-v3 for transcription, and SerpAPI for live web search inside the agent.
- Orchestration and infrastructure — n8n as the visual workflow engine, PostgreSQL (with the pgvector image, used as a plain relational store) for persistence, and Docker with Docker Compose for packaging and deployment. A Python script (deploy.py) performs the SSH/SFTP deployment to the server.

2.3. Fundamental Concepts

Several concepts recur throughout the report. A large-language model (LLM) is a neural network trained to predict text; given a conversation it produces a continuation, and with a system prompt and tools it can act as an agent. Retrieval-augmented generation (RAG) improves grounding by retrieving the most relevant chunks of a document — here ranked by cosine similarity over MiniLM embeddings — and inserting them into the prompt

before the model answers. Workflow orchestration means describing a multi-step pipeline (receive a request, branch on its type, enrich it, call the model, post-process, respond) as a graph that can be edited without redeploying application code; AstroBot uses n8n for this. JSON Web Tokens carry a signed, time-limited identity claim so the stateless API can authorise requests, and an `is_admin` claim gates the administrative service. Containerization packages each service with its dependencies so the system is reproducible from a fresh virtual machine.

3. Related Work

3.1. Existing Systems

General-purpose assistants such as ChatGPT, Claude, and Gemini provide strong conversational quality and, in their paid tiers, some multimodal features. However, advanced capabilities — large context windows, image generation, document analysis — are typically gated behind subscriptions, and the more specialised tasks (OCR, transcription, vector search over private documents, reproducible multi-step pipelines) are spread across separate products such as dedicated OCR services, speech-to-text APIs, and notebook-style RAG tools. Self-hosting alternatives (open models served locally) exist but require significant operational effort and rarely come with a polished, multi-feature interface out of the box.

3.2. Academic Studies

Research on retrieval-augmented generation has shown that grounding model outputs in retrieved passages reduces hallucination and improves factual accuracy on open-domain question answering, and that dense retrievers based on transformer embeddings with cosine similarity generally outperform sparse lexical baselines for this purpose. Work on LLM-based agents demonstrates that giving a model a small set of tools (search, a scratchpad for reasoning, a structured output contract) lets it decompose tasks reliably while keeping the surrounding system simple. On the engineering side, studies of agile software delivery report that iterative, feedback-driven development is well suited to projects whose requirements evolve as users try early versions — which is precisely the situation for a feature-rich assistant.

3.3. Comparative Analysis

Compared with the alternatives above, AstroBot emphasises three differentiators. First, breadth behind one surface: chat, image generation and editing, OCR and captioning, transcription, document RAG, notebook mode, PDF report generation, workflow chains, and an admin side are all reachable from a single application with a single account. Second, affordability: by combining free-tier model APIs with a self-hostable orchestration layer, the system is free to operate for the team and its users. Third, transparency and reproducibility: the orchestration logic is a visible n8n graph that can be tuned without redeploying code, the data model and deployment are documented, and the whole stack

comes up from a single docker compose command. AstroBot does not attempt to beat frontier models on raw quality; it integrates accessible models well.

4. Software Process Model

4.1. Selected Software Development Model

An iterative, Scrum-style agile model was selected. The set of desirable features for an AI assistant is large and evolves quickly as the team and informal testers use early versions, so a plan-driven waterfall process would have been a poor fit. The project was organised into four short sprints, each producing a working, testable increment of the product, with the most valuable and foundational items scheduled first.

4.2. Definition of Scrum Roles

Because the project was carried out by a four-person student team, the standard Scrum roles were mapped onto the team as shown in Table 4.1. The Product Owner role (prioritising the backlog, validating increments against acceptance criteria) was held primarily by the project lead in consultation with the advisor, who acted as the external stakeholder. The Scrum Master role (facilitating ceremonies, removing blockers) rotated informally but was anchored by the project lead. All four members formed the Development Team, each owning one or more feature domains.

Table 4.1: Scrum roles and the team members who held them.

Scrum role	Held by	Main responsibilities
Product Owner / Stakeholder Scrum Master	Tidiane Konaté	Backlog priority, acceptance of increments, scope decisions Sprint ceremonies, blocker removal, coordination
Development Team — Architecture / Backend / Deployment	Tidiane Konaté	System architecture, REST API, n8n workflow, Docker, deployment
Development Team — Vision / Product / UX	Sidi Mohamed Sall	Product framing, public interface design, usability sessions
Development Team — AI / LLM / Multimodal	Ahmed Essalem	Model integration, image/OCR/audio features, RAG pipeline
Development Team — QA / Frontend / Testing	Saad Ibrahim Houssein	Frontend polish, admin dashboard, integration test suite

4.3. Scrum Process

Each sprint followed the usual ceremonies adapted to the team's size: a sprint planning meeting to select backlog items and define acceptance criteria, short stand-up synchronisations to surface progress and blockers, a sprint review in which the increment was demonstrated, and a brief retrospective to adjust the way of working. The product

backlog was maintained as a list of user stories with priority and acceptance criteria; items not completed in a sprint were re-ranked for the next one. The advisor's review feedback fed directly into the backlog.

4.4. Scrum Implementation in the Project

The project was delivered in four sprints over roughly twelve weeks, as summarised in Table 4.2 and Figure 13.1 (in Appendix B). Sprint 1 established the foundations — repository, Docker Compose, database schema, JWT authentication, and a minimal chat path through n8n. Sprint 2 built the conversational core — the Mistral AI Agent, conversation history, personas, and the rich-content rendering and streaming-style reveal in the chat UI. Sprint 3 added the multimodal and retrieval features — image generation and image-to-image editing, OCR and BLIP captioning, Whisper transcription, RAG and notebook mode, and on-demand PDF reports. Sprint 4 delivered the administrative dashboard, the 66-case integration test suite, the deployment script, security hardening, and documentation.

Table 4.2: Sprint plan and delivered increments.

Sprint	Weeks	Goal	Delivered increment
1 — Foundations	≈ 1–3	Stand up the skeleton	Repo, Docker Compose, PostgreSQL schema, JWT auth, base chat via n8n
2 — Conversational core	≈ 3–5	Make the chat good	Mistral AI Agent, conversation history, 7 personas, Markdown/KaTeX/highlight.js/Mermaid, streaming-style reveal
3 — Multimodal & RAG	≈ 5–8	Add the AI features	Image generation & img-to-img, OCR, BLIP captions, Whisper transcription, RAG & notebook mode, PDF report generation, workflow chains, tags/exports
4 — Admin, tests & deploy	≈ 8–11	Operate & harden	Admin dashboard (stats, users, audit), 66-case integration test suite, deploy.py, security hardening, documentation

5. Requirements Analysis

5.1. Stakeholders

The primary stakeholders are the end users of the public application — students preparing reports and code, developers exploring multimodal AI, and knowledge workers who need question answering over their own documents — and the administrator, who operates the deployment, manages accounts, and audits usage. Secondary stakeholders include the project advisor, who acted as the external reviewer and shaped the scope, and the development team itself, which relied on a reproducible build and a fast deployment loop.

5.2. User Requirements

Requirements were gathered from the team's own use of comparable tools, short informal usability sessions with fellow students, and the advisor's reviews, then translated into prioritised backlog items with acceptance criteria and validated at sprint reviews. They are summarised below as functional and non-functional requirements.

5.2.1. Functional Requirements

The functional requirements are summarised in Table 5.1.

Table 5.1: Functional requirements.

ID	Functional requirement
FR-1	A visitor can register an account; authenticated users can log in and out.
FR-2	A user can send a text message and receive a model reply, with the last fifteen messages of the session kept as context.
FR-3	A user can choose a persona that adjusts the assistant's tone.
FR-4	A user can request an image from a text prompt and receive it inline in the conversation.
FR-5	A user can upload an image and ask the assistant to edit it (image-to-image).
FR-6	A user can attach an image and have its text extracted (OCR) and/or described (captioning), with the result used as context.
FR-7	A user can upload a PDF and ask questions answered from its content (RAG), or pin it as durable notebook context.
FR-8	A user can record audio and have it transcribed to text.
FR-9	A user can obtain a generated PDF report of a typed response.
FR-10	A user can organise sessions with colour-coded tags, search the conversation history, and export a session as PDF, Markdown, or JSON.
FR-11	A user can define and replay reusable multi-step workflow chains.
FR-12	A user can manage their profile and avatar (with an in-browser cropper).
FR-13	An administrator can log in to a separate dashboard and view real-time statistics and activity charts.
FR-14	An administrator can search, suspend, reactivate, reset the password of, delete, and create user accounts.
FR-15	An administrator can browse any user's conversation history grouped by session.

5.2.2. Non-Functional Requirements

Table 5.2: Non-functional requirements.

ID	Category	Requirement
NFR-1	Performance	Core chat and CRUD requests shall complete within an interactive latency budget under normal load; long model calls are bounded by the n8n agent's iteration cap.
NFR-2	Security	Passwords shall be stored hashed with bcrypt (12 salt rounds); sessions shall use signed JWTs with a 7-day expiry; the admin service shall reject any token without an <code>is_admin</code> claim.
NFR-3	Security	HTTP responses shall carry Helmet security headers; rate limiting shall apply (auth 20/15 min, chat 30/min, admin 500/15 min); uploads shall be capped at 10 MB and 5 attachments per message.
NFR-4	Reliability	The application shall self-heal on restart through idempotent database migrations and an admin re-seed; containers shall use restart policies so a host reboot recovers the system.
NFR-5	Maintainability	Orchestration logic shall live in a visible n8n workflow editable without redeploying code; the codebase shall be modular by feature domain.
NFR-6	Portability	The whole system shall come up from a single docker compose command and shall be deployable to a fresh virtual server with one script.
NFR-7	Cost	The system shall rely only on free-tier external APIs and self-hostable components.

5.3. System Specifications

The backend exposes a REST API: authentication and profile endpoints under `/api/auth`, chat and history endpoints under `/api/chat`, and the document, RAG, transcription, image-edit, YouTube, and workflow endpoints under `/api/features`; the admin service exposes a parallel `/api` set. Selected endpoints are listed in Table 5.3. Every chat message is forwarded to the n8n webhook, which returns a JSON object carrying the reply text and an optional action (`generate_image` or `generate_pdf`). The persistent data entities are users, conversations (with optional image, PDF, and attachment fields), documents, document_chunks (with embeddings stored as text), notebook_bindings, and workflows; they are detailed in Section 7.

Table 5.3: Selected REST endpoints and their responsibilities.

Method & path	Service	Responsibility
POST <code>/api/auth/register</code> · <code>/login</code>	Main	Account creation and login; issues a 7-day JWT.
PUT <code>/api/auth/profile</code> · <code>/password</code> · <code>/avatar</code>	Main	Update profile fields, change password, set/clear avatar.
POST <code>/api/chat/message</code>	Main	Core chat: OCR/BLIP on images, history + persona enrichment, n8n round-trip, image/PDF generation, persistence.
GET <code>/api/chat/history</code> · <code>/sessions</code> · <code>/session/:id</code>	Main	Conversation history, distinct sessions, and a single session's messages.
PUT <code>/api/chat/sessions/:id/tag</code> · GET <code>/tags</code>	Main	Folder/tag system over sessions.
GET <code>/api/chat/export/:id?format=pdf md json</code>	Main	Export a session in the chosen format.
POST <code>/api/features/documents</code> · PUT	Main	Upload a PDF (chunk + embed), bind it to a

/notebook		session in notebook or RAG mode.
POST /api/features/transcribe · /image-edit · /youtube-transcript	Main	Audio transcription (Whisper), image-to-image edit, YouTube summary.
GET/POST/PUT/DELETE /api/features/workflows	Main	CRUD for user-defined workflow chains.
GET /api/stats	Admin	Dashboard metrics: user/message/session/token/image/PDF/doc totals, daily activity, top users.
GET/POST/PUT/DELETE /api/users[:id]	Admin	List, create, update (status/role), reset password, delete users; per-user conversations.

6. System Modeling

6.1. Context Diagram

Figure 6.1 shows the context model. The AstroBot platform (the public web app, the admin service, and the n8n orchestrator) is the system of interest. Two human actors interact with it: the end user, who sends chat messages, uploads files, and consumes replies and generated media, and the administrator, who manages accounts and reviews statistics. The platform integrates with several external systems: Mistral Cloud for LLM inference, the HuggingFace Inference router for image generation, captioning and embeddings, Groq Whisper for transcription, SerpAPI for live web search, and a shared PostgreSQL database for persistence.

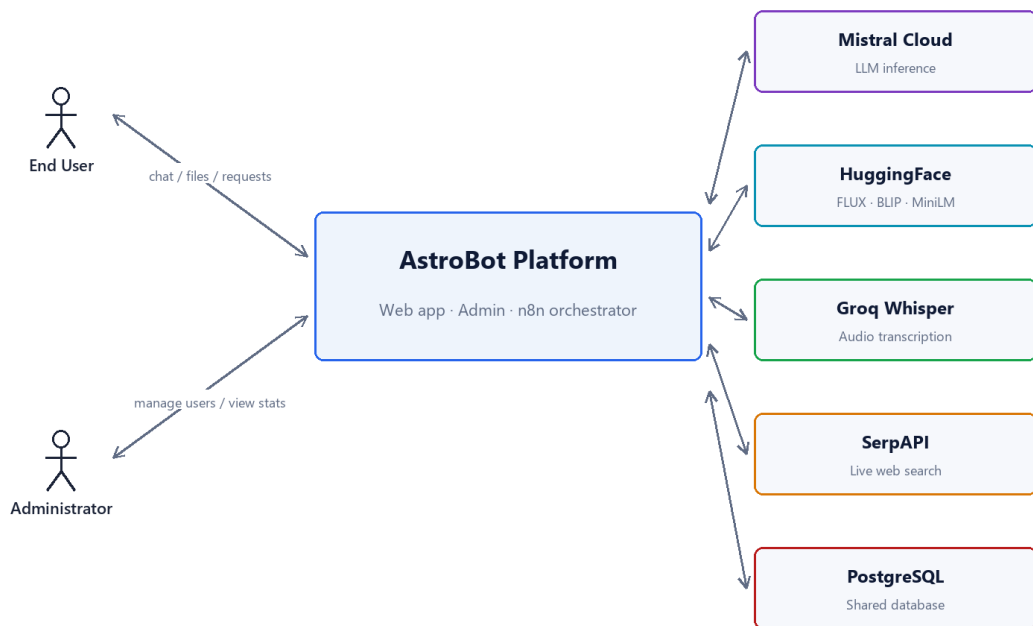


Figure 6.1: Context diagram of the AstroBot platform.

6.2. Use Case Diagram

Figure 6.2 summarises the use cases. The end user can register and log in, send chat messages, generate and edit images, analyse images via OCR and captioning, ask questions over a PDF using RAG, transcribe audio, generate PDF reports, run saved workflow chains, export conversations, and manage their profile and avatar. The administrator

additionally reviews the statistics dashboard, manages user accounts (search, suspend, reset, delete, create), and audits conversation history.

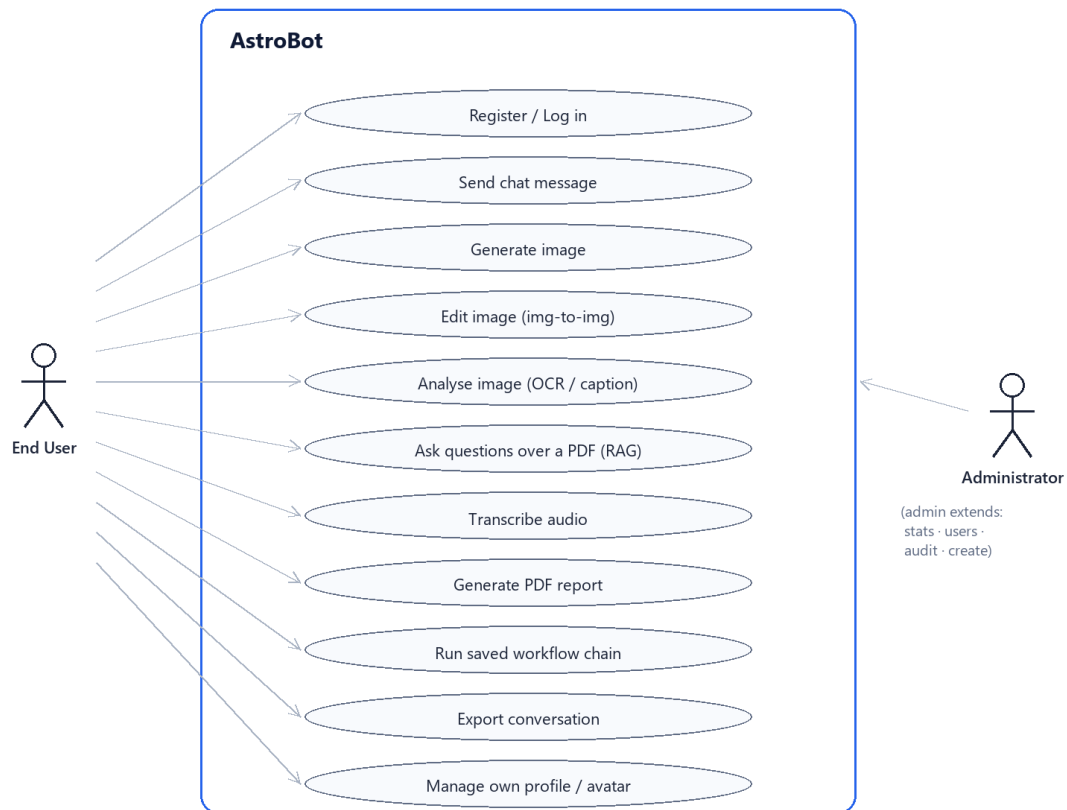


Figure 6.2: Use-case diagram (end user and administrator).

6.3. Use Case Descriptions

Two representative use cases are described below.

Send chat message — Actor: authenticated end user. Preconditions: the user holds a valid JWT and is on the chat page. Main flow: the user types a message, optionally attaches images, and submits; the backend runs OCR and captioning on any images, loads the last fifteen messages of the session, prepends the active persona directive and any notebook/RAG context, and forwards the enriched request to the n8n webhook; the n8n AI Agent calls Mistral (with optional SerpAPI search and a reasoning scratchpad) and returns a JSON reply; the backend triggers image or PDF generation if the reply requests it, persists the conversation row, and returns the reply and any media. Postconditions: the conversation history contains the new exchange. Alternative flows: if the model output cannot be parsed as the expected contract the backend falls back to treating the whole output as the reply text; rate-limit and validation errors return clear 4xx responses.

Ask questions over a PDF (RAG) — Actor: authenticated end user. Preconditions: the user has uploaded a PDF and bound it to the session in RAG mode. Main flow: on each question the backend embeds the question with MiniLM, retrieves the four most similar chunks of the document by cosine similarity, inserts them into the prompt, and proceeds as in the chat use case. Postconditions: the answer is grounded in the retrieved chunks. Alternative flow: in notebook mode the first six thousand characters of the document are used as context instead of retrieved chunks.

6.4. Sequence Diagrams

Figure 6.3 traces a chat message round-trip. The browser posts the message (with any attachments and the JWT) to the main app, which loads the session history from PostgreSQL, posts an enriched prompt to the n8n webhook, where the AI Agent calls Mistral (and, when needed, HuggingFace for images or embeddings); n8n returns a structured JSON response, the main app generates an image or PDF if requested, inserts the conversation row, and returns the reply, media, and metadata to the browser.

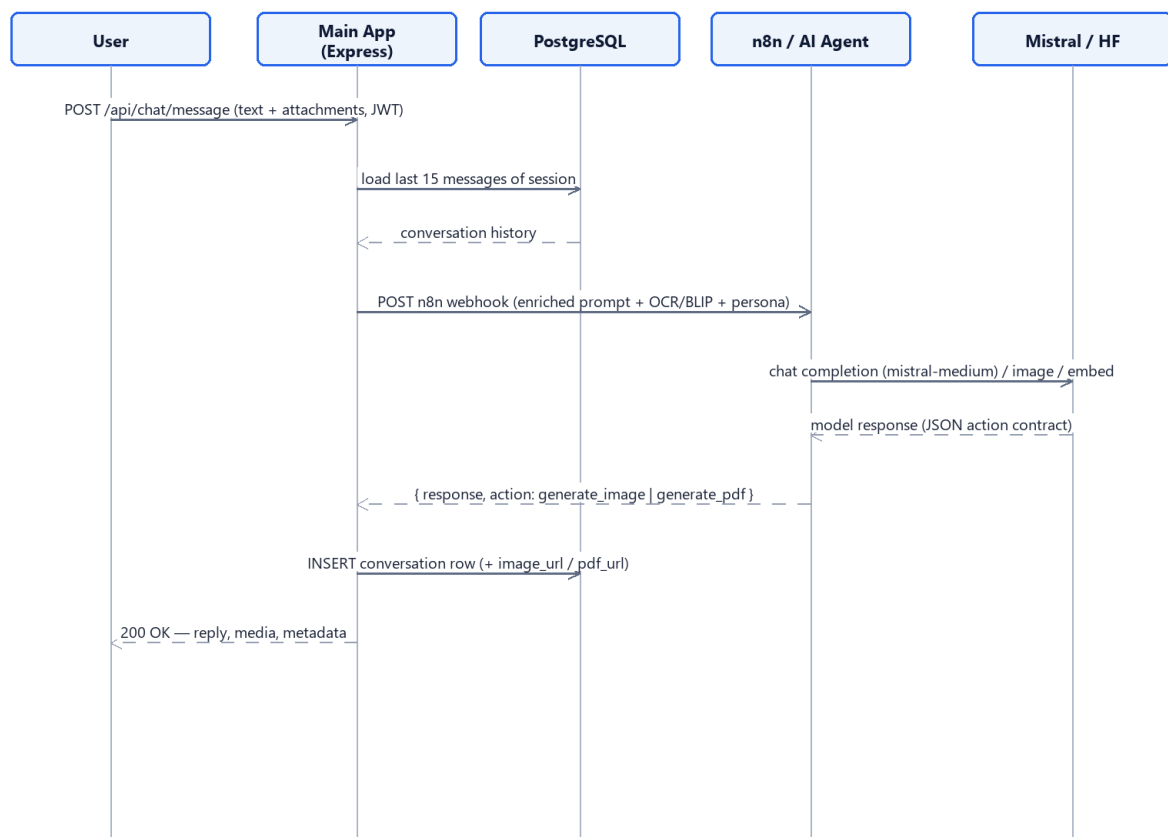


Figure 6.3: Sequence diagram for a chat message round-trip.

7. System Architecture and Design

7.1. System Architecture

AstroBot follows a layered client–server architecture, shown in Figure 7.1, deployed as three Docker services on one private network. The client tier is the browser, serving the public chat UI and the admin dashboard UI from static assets. The application tier contains two independent Express services: the main app on port 3000 (public chat, JWT-authenticated, with Helmet and rate limiting) and the admin service on port 7040 (gated by an `is_admin` claim, completely separate from the public process). The orchestration tier is an `n8n` instance on port 5678 that hosts the AI Agent workflow and reaches Mistral, SerpAPI, a Think scratchpad, and a self-hosted Whisper transcriber. The data and external tier consists of a shared PostgreSQL container (used as a plain relational store), the HuggingFace Inference router (FLUX, BLIP, MiniLM, with a `fal-ai` fallback), and the external Mistral, Groq, and SerpAPI services. This shape was chosen so that the existing PostgreSQL container can be reused without operating a second database, all AI traffic exits through `n8n` so prompts and tools can be tuned without redeploying, and a public-facing exploit can never reach the admin process.

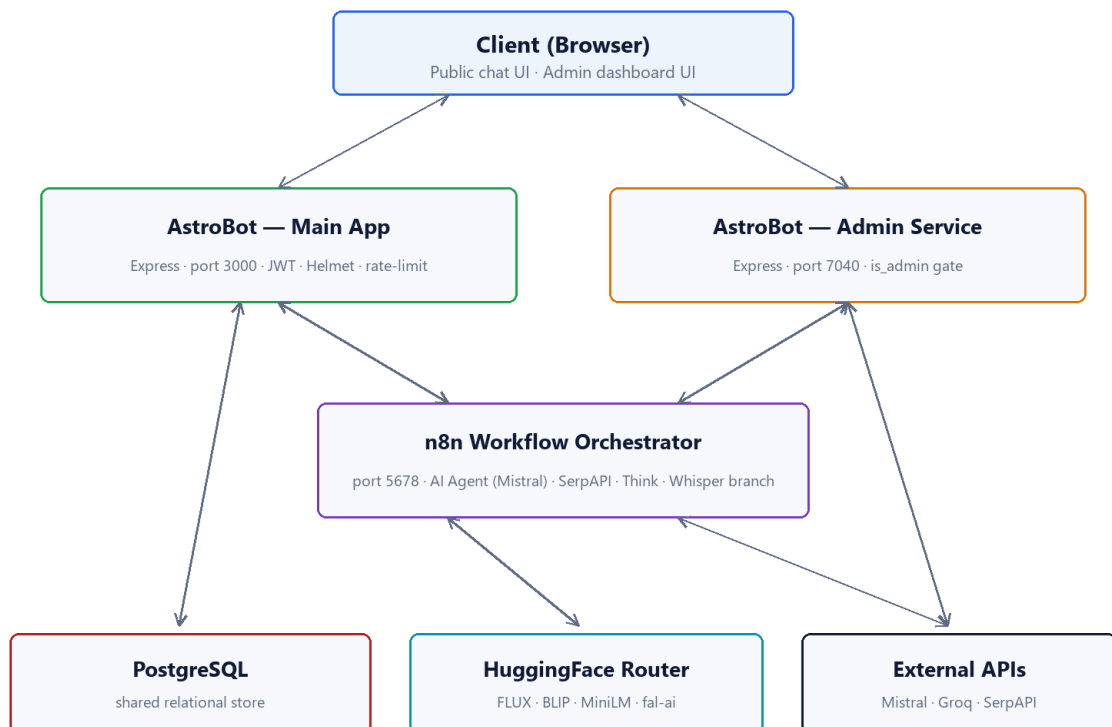


Figure 7.1: System architecture (client / application / orchestration / data tiers).

7.2. Structural Models

The static design is centred on the persistent data model rather than a deep class hierarchy: the backend is organised by feature modules (auth, chat, features) with controller, service, and data-access responsibilities, and the durable state lives in a small set of relational tables described below and in Figure 7.2.

Table 7.1: Main database tables and their purpose.

Table	Purpose
users	Accounts: name, email, hashed password, is_admin, status, avatar, last_login.
conversations	One row per message/response, with session_id, session_tag, and optional image_url, pdf_url, pdf_filename, and an attachment JSON blob.
documents	Uploaded files for notebook/RAG: name, mime, size, extracted text_content.
document_chunks	Per-document chunks for RAG: chunk_index, chunk_text, and an embedding stored as text.
notebook_bindings	Which document is active for which session, and the mode (notebook rag).
workflows	User-defined multi-step prompt chains, stored as a steps JSON array.

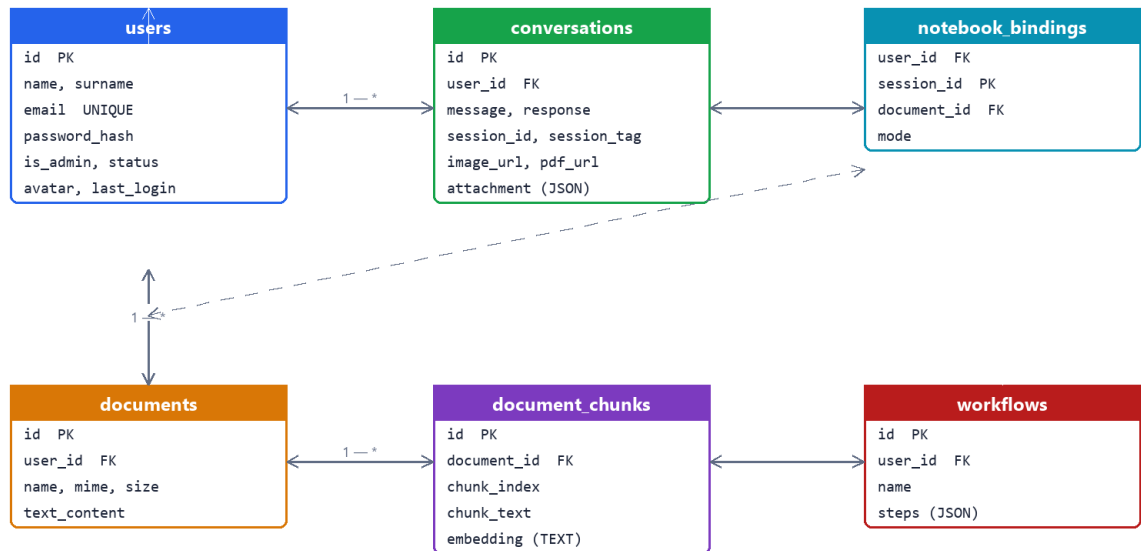


Figure 7.2: Entity-relationship model of the PostgreSQL schema.

7.2.1. Class / Entity-Relationship Diagram

Because the backend is JavaScript with direct SQL access rather than an object-oriented domain model, the structural design is best expressed as the entity-relationship diagram in Figure 7.2. A user owns many conversations, many documents, many notebook bindings, and many workflows (one-to-many in each case); a document owns many document chunks (one-to-many); and a notebook binding links a user's session to one document. Foreign keys cascade on delete so removing a user or a document removes its dependent rows.

7.2.2. Inheritance (Generalization)

The system uses very little inheritance by design. Conceptually the user type is specialised into a regular user and an administrator, but this specialisation is represented by a boolean `is_admin` flag on the single users table rather than by a subclass, which keeps the schema and the authorisation logic simple — the admin service simply requires `is_admin` to be true in the JWT.

7.2.3. Aggregation / Composition

Aggregation and composition appear in two places. A conversation session aggregates its individual message rows (they share a `session_id` but each row is independently meaningful), and a document composes its chunks: the chunks have no meaning outside their parent document and are deleted with it (composition, enforced by an `ON DELETE CASCADE` foreign key). Notebook bindings are association objects linking a session and a document with a `mode` attribute.

7.3. Behavioral Models

The dynamic design is event-driven at the orchestration boundary and state-based for an individual message.

7.3.1. Event-Driven Model

Every chat message is an event posted to the n8n webhook. Inside the workflow a Switch node routes the event by type (audio vs. text); the audio branch posts the file to a Whisper transcriber and rejoins the text branch; a code node enriches the payload with the user's name and conversation history; the AI Agent node consumes it and may itself emit tool-call events to SerpAPI (capped at two searches) and to a Think scratchpad before producing its JSON reply, which a final node extracts and returns. On the client, the bot

reply is revealed word by word, an animation driven by a timer rather than by server-side streaming.

7.3.2. State Diagram

Figure 7.3 models the lifecycle of a single conversation message. From the Idle state, the user sending a message moves the session into Processing; if the model needs a tool or media generation the session enters Awaiting tool / media and returns to Processing when the result arrives; when the answer is ready the session moves to Replied; sending the next message returns it to Processing, and closing the session is the terminal transition.

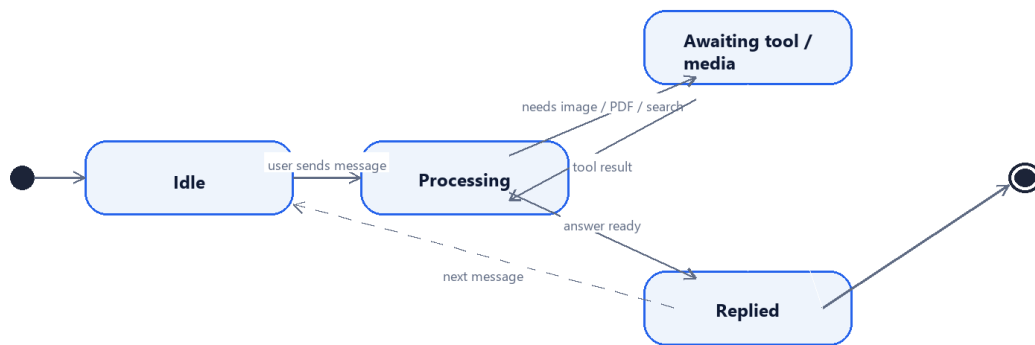


Figure 7.3: State diagram of a conversation message.

8. System Implementation

8.1. Development Environment

Development used Git for version control with a public mirror on GitHub (<https://github.com/tkonate788/astrobot>), Visual Studio Code as the editor, Node.js 18 for the backend, and Docker Desktop / Docker Compose for running the services locally exactly as they run in production. The n8n workflow was developed in the n8n editor and exported as JSON kept in the repository. Deployment to the server is performed by a Python script (deploy.py) that packages the project, uploads it over SFTP, and runs docker compose up -d --build remotely; a 66-case Python integration suite (test_all.py) is run against the live deployment.

8.2. System Components

The MVP comprises four components. The main application (frontend + backend) serves the public chat UI and the REST API on port 3000. The admin service serves the dashboard UI and a parallel API on port 7040. The n8n instance on port 5678 hosts the orchestration workflow. The shared PostgreSQL container provides persistence. Beyond these, the system depends on several third-party services, summarised in Table 8.1. Figure 8.1 shows the n8n workflow that every message passes through.

Table 8.1: Third-party services used by AstroBot.

Service	Used for	How it is reached
Mistral Cloud (mistral-medium-latest)	Conversational LLM and the agent's reasoning	Inside the n8n AI Agent node
HuggingFace Inference router	Image generation (FLUX.1-schnell), image captioning (BLIP), sentence embeddings (MiniLM-L6-v2)	HTTPS from the backend / n8n
fal-ai	Fallback image generation and image-to-image editing	HTTPS from the backend
Groq (Whisper-large-v3-turbo)	Audio transcription	HTTPS from the backend / n8n audio branch
SerpAPI	Live web search available to the agent (capped at two calls per message)	Inside the n8n AI Agent as a tool
PostgreSQL (shared container)	Persistent storage for all entities	TCP on the private Docker network

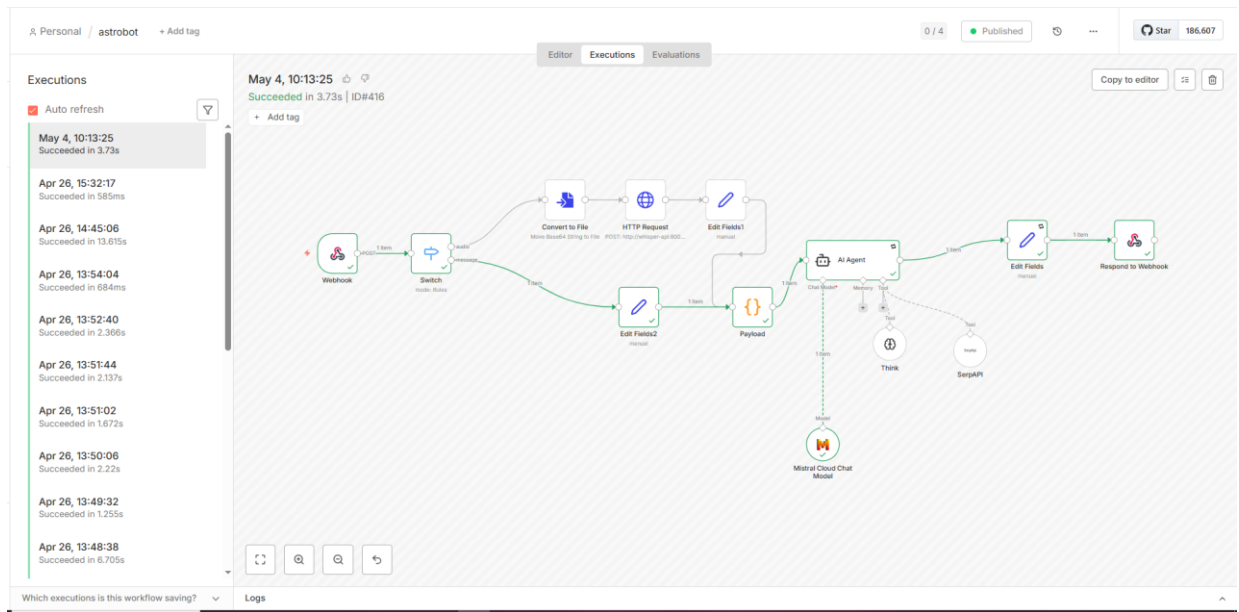


Figure 8.1: n8n workflow that routes every message through Mistral, with optional web search and reasoning tools.

8.3. Code Structure

The repository is organised by component and, within the backend, by feature domain:

```
astrobot/
├── frontend/
│   ├── index.html chat.html login.html register.html
│   ├── style.css script.js
│   └── team/
│       └── # developer photos for the About section
├── backend/
│   ├── server.js
│   │   └── # Express entry point, middleware, routes
│   ├── db/index.js
│   │   └── # PostgreSQL pool + idempotent migrations + admin seed
│   ├── middleware/auth.js
│   │   └── # JWT verification
│   └── routes/
│       ├── auth.js
│       │   └── # register / login / profile / avatar
│       ├── chat.js
│       │   └── # messages, image gen, PDF, history, sessions, export
│       └── features.js
│           └── # documents, RAG, notebook, transcribe, YouTube, img-edit,
├── workflows
├── admin/
│   ├── backend/server.js
│   │   └── # separate Express service (port 7040)
│   └── backend/public/
│       └── # admin dashboard UI (vanilla JS + Chart.js)
├── docker/
│   ├── docker-compose.yml
│   │   └── # the two AstroBot services on the n8n network
│   ├── Dockerfile
│   │   └── # main app
│   └── .env.example
│       └── # template for local secrets
├── deploy.py
│   └── # SSH + SFTP + docker compose deployment
├── test_all.py
│   └── # 66-case integration test suite
└── README.md
```

Within the backend, each route module separates request handling from the operations it performs, and the database module owns all SQL and the schema migrations, so the schema can evolve safely on every redeploy. Representative code excerpts are given in Appendix A.

9. MVP Development

9.1. MVP Definition

The MVP is defined as the smallest deployable system that delivers real value to end users and to an administrator: a working chat assistant with conversation memory, the multimodal and retrieval features listed below, persistent and exportable conversations, and a separate admin dashboard — all running on a real server and covered by an automated test suite. Anything beyond that (native mobile apps, billing, a fine-tuned model) is explicitly future work.

9.2. Implemented Features

The MVP implements the following capabilities:

- Conversational chat — Mistral medium-latest via the n8n AI Agent, with a fifteen-message context window per session and seven selectable personas.
- Rich content rendering — Markdown, KaTeX mathematics, highlight.js code highlighting, and Mermaid diagrams in the bot bubble, with a word-by-word reveal.
- Image generation — FLUX.1-schnell through the HuggingFace router with a fal-ai fallback, plus image-to-image editing (style transfer) via FLUX dev.
- OCR and visual understanding — Tesseract.js OCR (English and French) and BLIP captioning, with both signals added to the prompt before the model answers.
- Document Q&A — retrieval-augmented generation over uploaded PDFs (chunking, MiniLM embeddings, top-4 cosine retrieval) and a notebook mode that pins the document as durable context.
- Audio transcription — browser recording transcribed by Groq Whisper-large-v3.
- PDF report generation — typed responses turned into a downloadable PDF (PDFKit), with a heuristic that forces generation when the model acknowledges a report.
- Productivity tools — colour-coded session tags and search, conversation export (PDF / Markdown / JSON), a YouTube summariser, reusable workflow chains, and a profile/avatar editor with an in-browser cropper.
- Administration — a separate dashboard with real-time KPIs and charts, user management (search, suspend, reactivate, reset password, delete, create), and per-user conversation auditing grouped by session.

9.3. System Interfaces

The public application opens on a landing page (Figure 9.1) with a central chat bar and a friendly mascot, a features section (Figure 9.2), and an about page with the team (Figure 9.3). After logging in the user reaches the chat interface (Figure 9.4) with the session sidebar and attachment controls; a persona selector (Figure 9.5) adjusts the assistant's tone, and a profile panel with an in-browser image cropper (Figure 9.6) manages the account. The administrator uses a separate dashboard with KPI tiles and activity charts (Figure 9.7), a user-management table (Figure 9.8), and a self-service settings panel (Figure 9.9).

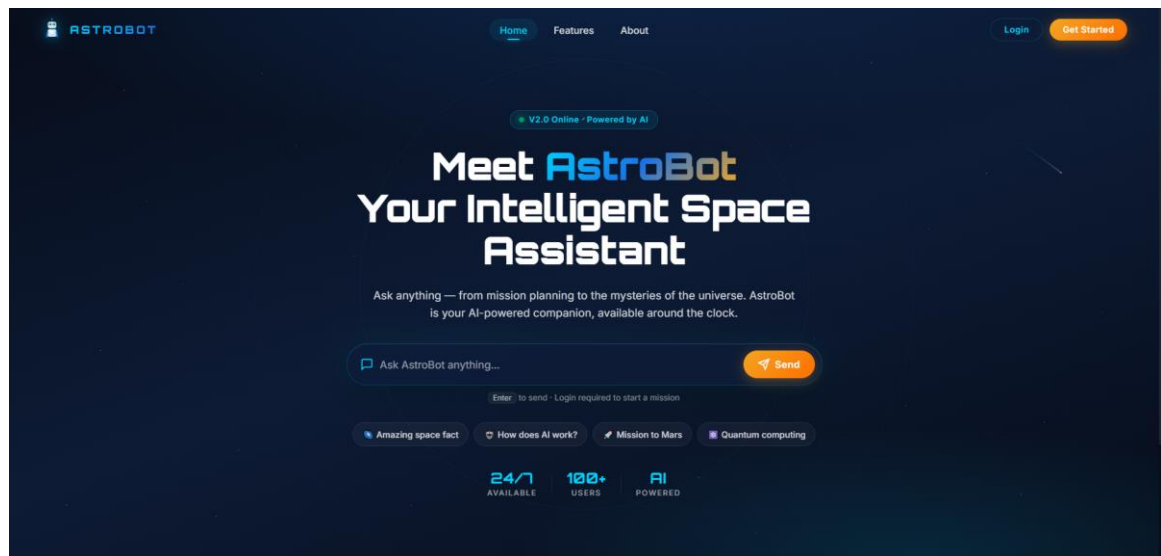


Figure 9.1: Public landing page with the central chat bar.

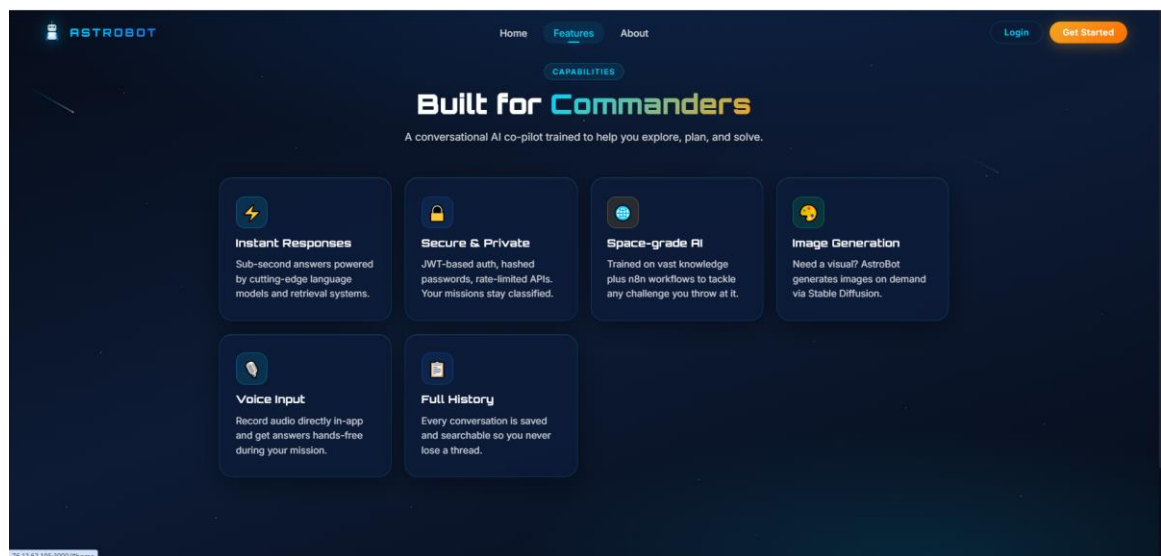


Figure 9.2: Features section of the public application.

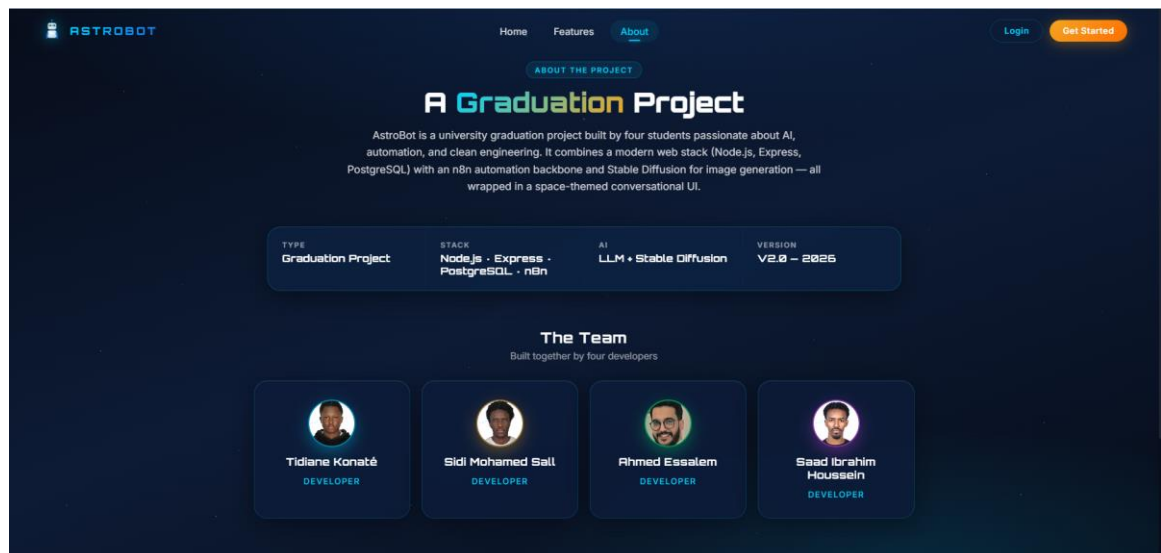


Figure 9.3: About page with the four-developer team.

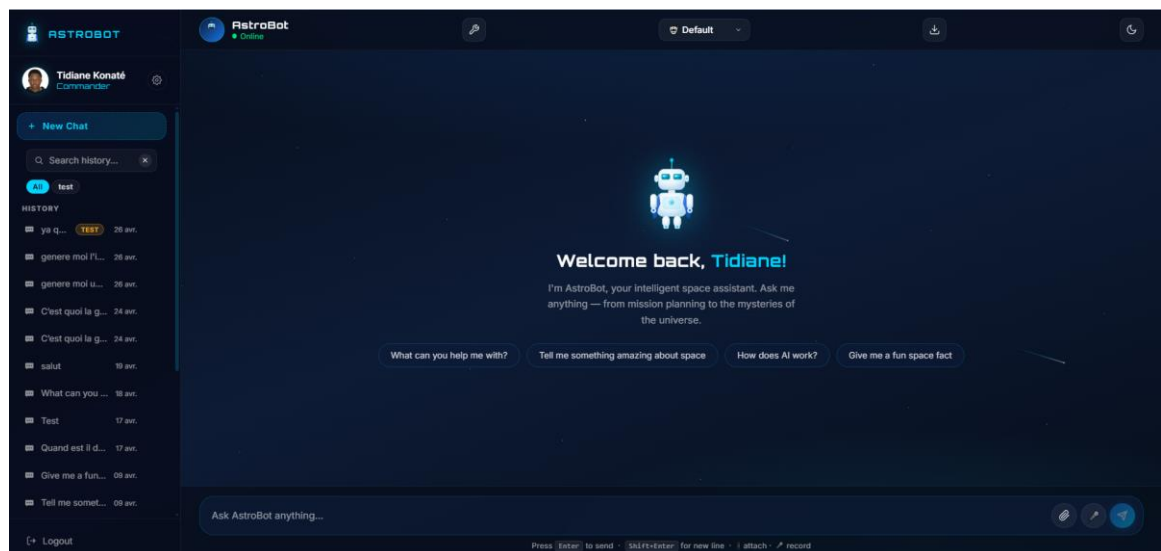


Figure 9.4: Authenticated chat interface (welcome screen).

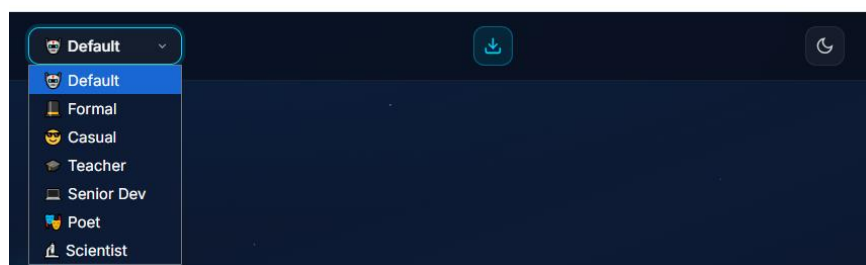


Figure 9.5: Persona selector — seven selectable assistant personas.

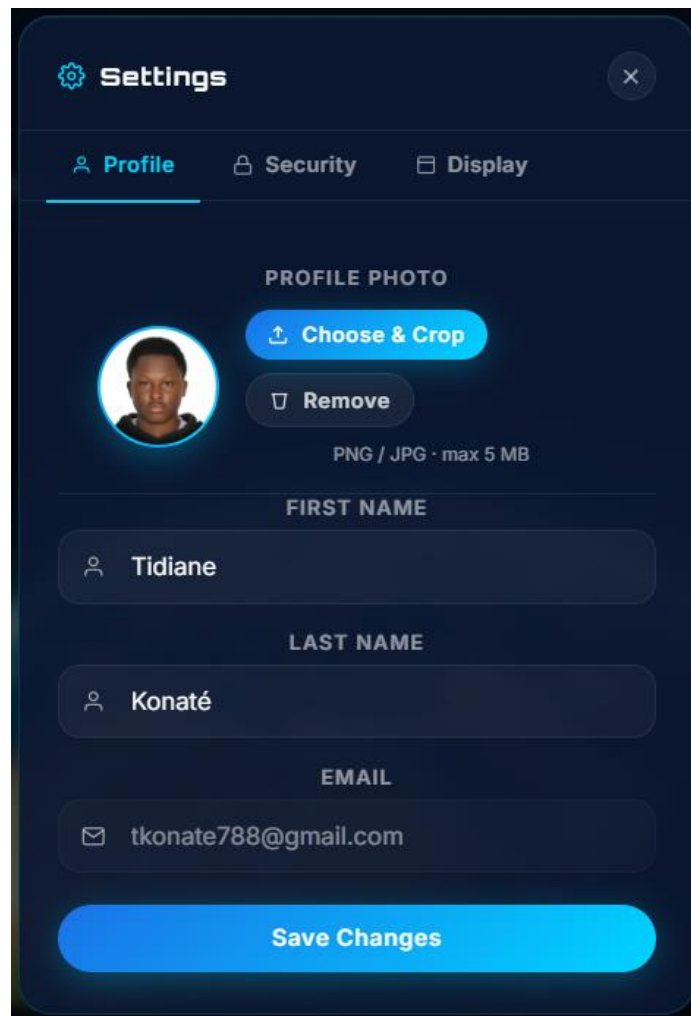


Figure 9.6: Profile settings with an in-browser image cropper.

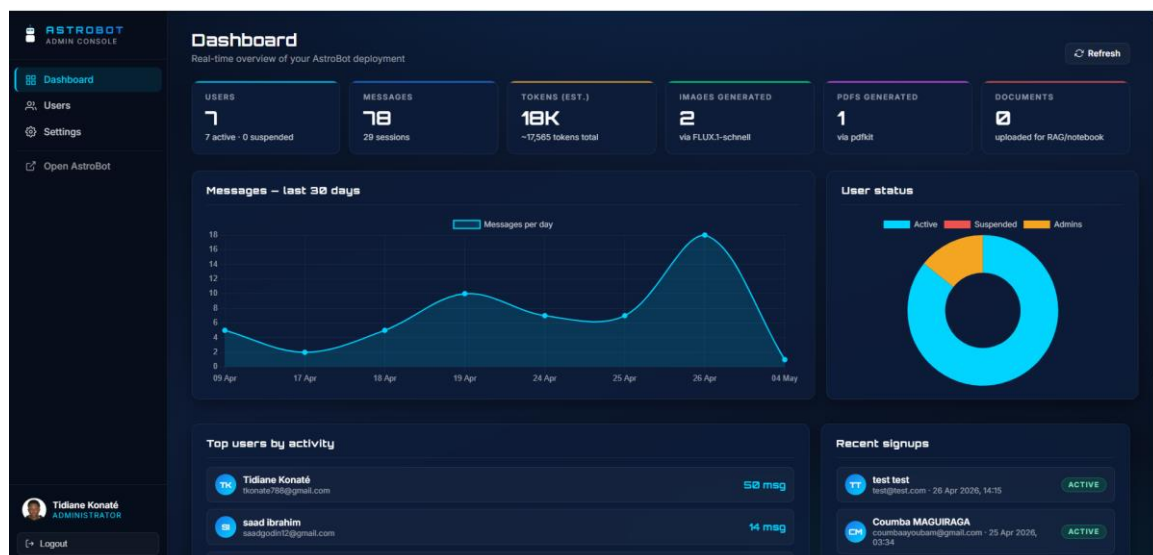


Figure 9.7: Admin dashboard — KPIs and activity charts.

USER	EMAIL	STATUS	ROLE	SESSIONS	MESSAGES	LAST LOGIN	JOINED
test test	test@test.com	ACTIVE	USER	0	0	—	26 Apr 2026, 14:15
Coumba MAQUIRAGA	coumbaayoubam@gmail.com	ACTIVE	USER	1	2	—	25 Apr 2026, 03:34
Sidi Mohamed Sall	sidimohamedsall5@gmail.com	ACTIVE	USER	1	5	—	25 Apr 2026, 03:34
Mamadou Keita	mkelta5912@gmail.com	ACTIVE	USER	2	6	—	24 Apr 2026, 14:21
saad Ibrahim	saadgodin12@gmail.com	ACTIVE	USER	4	14	—	18 Apr 2026, 20:04
Modi Niang	modiniang1975@gmail.com	ACTIVE	USER	1	1	—	2 Apr 2026, 00:42
Tidiane Konaté	tkonate788@gmail.com	ACTIVE	ADMIN	20	50	4 May 2026, 10:20	12 Mar 2026, 15:54

Figure 9.8: Admin user-management table.

Profile photo

Choose & Crop Remove

Profile info

FIRST NAME

Tidiane

LAST NAME

Konaté

EMAIL

tkonate788@gmail.com

Save

Change password

CURRENT PASSWORD

NEW PASSWORD

CONFIRM NEW PASSWORD

Update password

Figure 9.9: Admin self-service settings (profile and password).

9.4. System Demonstration

The features were demonstrated against the live deployment with representative scenarios: a Markdown-rich answer rendered in the chat bubble (Figure 9.10); generating an image from a prompt (Figure 9.11); turning a typed response into a downloadable PDF report (Figure 9.12); attaching a screenshot so the assistant reads its text via OCR and explains it (Figure 9.13); uploading a PDF and binding it as notebook/RAG context (Figure 9.14); and composing a reusable workflow chain (Figure 9.15). Audio transcription was demonstrated by recording a short clip and receiving the transcript inline.

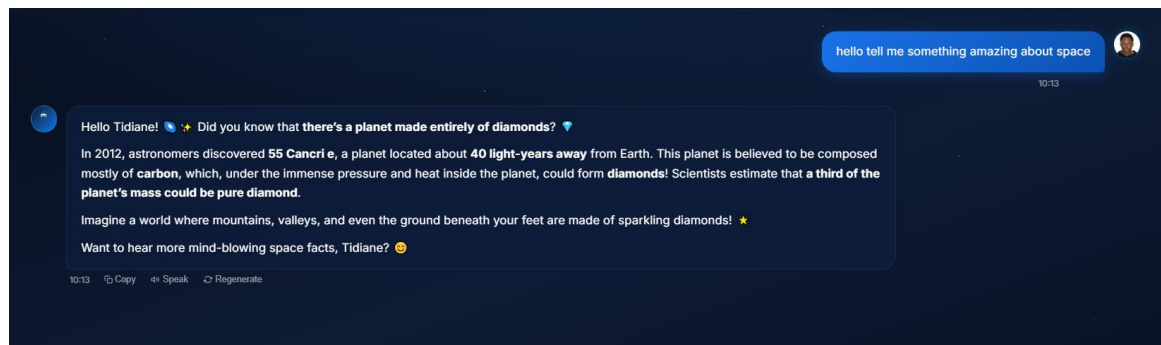


Figure 9.10: Markdown-rich reply rendered inside a bot bubble.

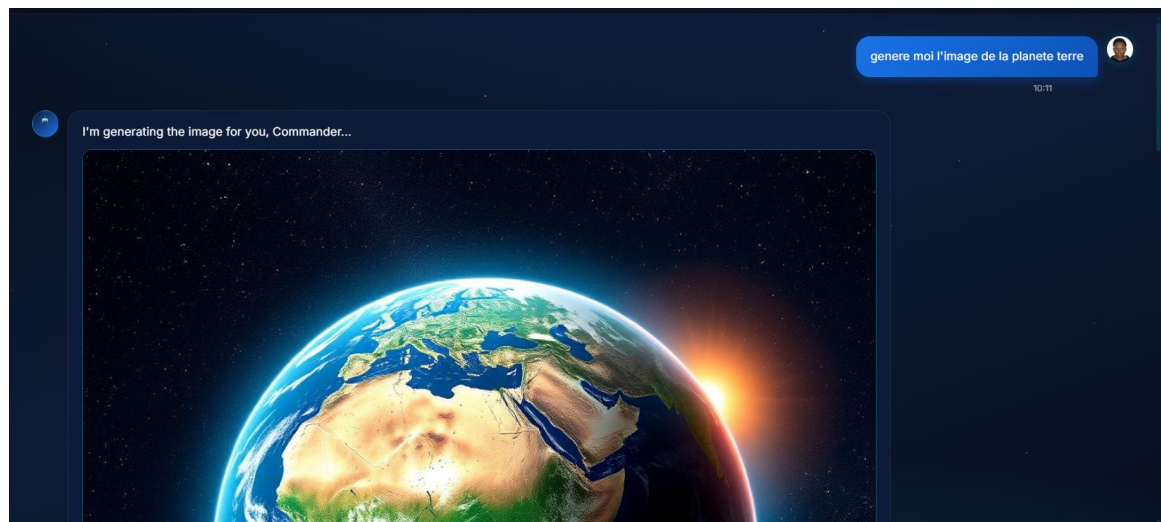


Figure 9.11: AI image generation example.

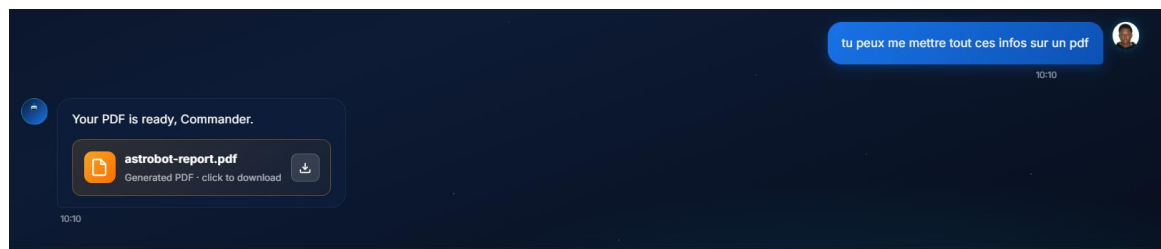


Figure 9.12: On-demand PDF report shown as a download card.

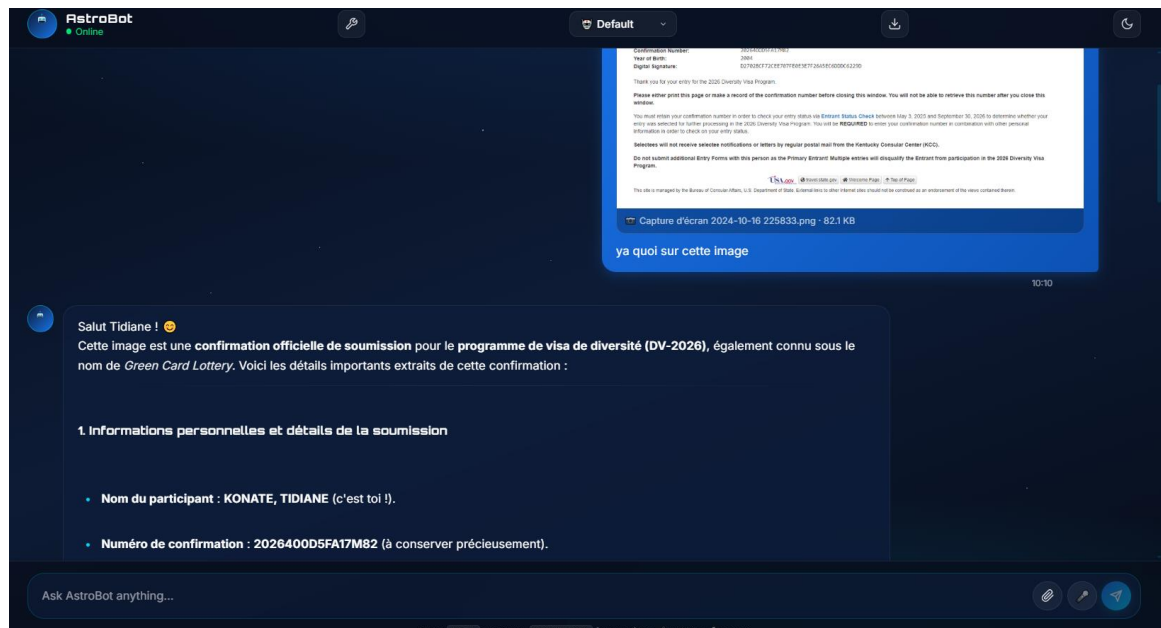


Figure 9.13: OCR analysis of an attached image.

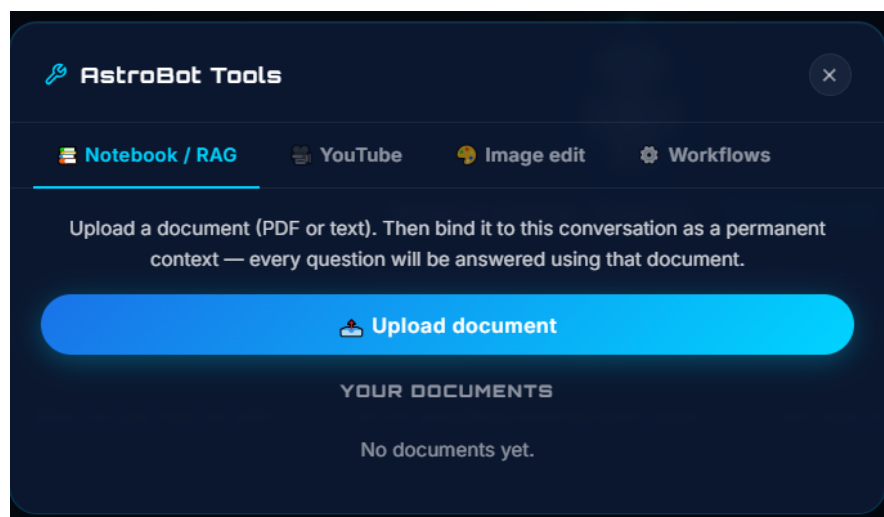


Figure 9.14: Document Q&A — uploading and binding a PDF (notebook / RAG).

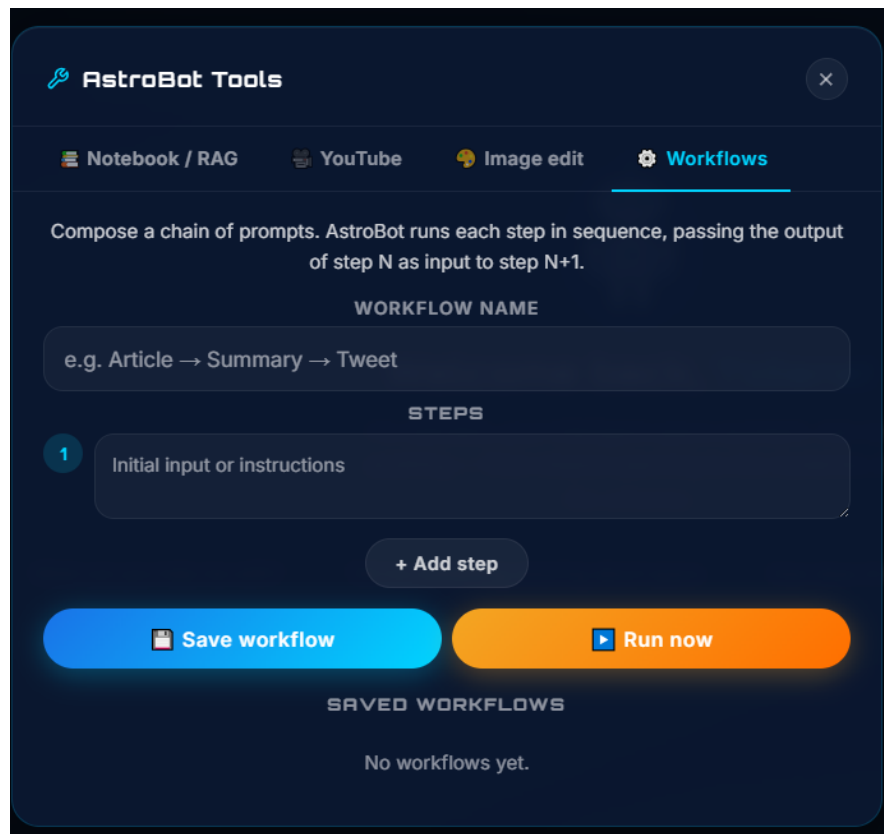


Figure 9.15: Composing a reusable multi-step workflow chain.

10. Testing and Evaluation

10.1. Testing Strategy

Testing combines integration testing of the running system with scenario-based acceptance checks. The primary instrument is `test_all.py`, a Python suite of 66 cases that runs against the live deployment (main app on port 3000 and admin service on port 7040), exercising real HTTP requests, the database, and the n8n round-trip rather than mocks. In addition, each backlog item was checked against its acceptance criteria at the corresponding sprint review, and informal usability sessions surfaced interface refinements.

10.2. Test Scenarios

Representative scenarios covered by the suite include: service health of both applications; static assets served with the correct no-cache headers; authentication flows (correct and wrong credentials, missing token, role check); the full admin CRUD surface (statistics, list, create, update, suspend/activate, reset-password, delete); main-app profile and avatar operations and password change; the chat round-trip through n8n and Mistral; session tags, history, session detail and deletion; all three export formats (PDF, Markdown, JSON); the document, RAG bind/unbind, and workflow-CRUD endpoints; and input validation (malformed requests returning clean 4xx responses rather than 5xx).

10.3. Test Results

All 66 cases pass against the live deployment. Table 10.1 summarises the results by category. The few issues discovered during development were addressed before this report — most notably stale static assets on iOS Safari (fixed with no-store cache headers) and the model occasionally acknowledging a PDF report without flagging the action (handled by re-parsing the reply and forcing generation). The main limitation of the evaluation is scale: testing was functional and small-scale rather than a load test, which is left to future work.

Table 10.1: Integration test suite — results by category.

Category	Result
Service health (main + admin)	passed
Static assets & no-cache headers	passed
Authentication & role checks	passed
Admin CRUD (stats, users, lifecycle)	passed
Main-app profile / avatar / password	passed
Chat round-trip (n8n → Mistral)	passed
Sessions, tags, history, deletion	passed

Exports (PDF / Markdown / JSON)	passed
Documents, RAG bind/unbind, workflows	passed
Input validation (clean 4xx)	passed
Total	66 / 66 passed

11. Results and Discussion

The project produced a working, deployed Minimum Viable Product that meets its objectives: a single web application that brings conversational AI together with image generation and editing, OCR and captioning, audio transcription, retrieval-augmented question answering, notebook context, PDF report generation, and reusable workflow chains, plus a separate administrative dashboard — all built from free-tier APIs and self-hostable components, packaged with Docker, deployed on a live server, and covered by a 66-case integration test suite that passes in full. The Scrum-style process worked well for this kind of project: scheduling the foundations and the conversational core first meant every later sprint built on a stable base, and the advisor's reviews fed directly into the backlog. Routing all AI traffic through a visible n8n workflow proved valuable in practice — prompts, tool limits, and the audio branch were tuned several times without redeploying application code. The main limitations are the breadth-over-depth trade-off (the system integrates accessible models rather than competing with frontier ones), reliance on the availability and rate limits of free external services, and the small scale of the evaluation.

12. Conclusion and Future Work

12.1. Conclusion

AstroBot demonstrates a complete software-engineering pipeline — requirements elicitation, modelling, architecture and design, implementation, testing, and deployment — applied to a practical problem: the fragmentation of everyday AI tooling. The resulting MVP is live, reproducible, and useful, and it establishes a clean baseline architecture (a public app, a separate admin service, and an n8n orchestration layer over a shared database) on which further capabilities can be added incrementally.

12.2. Future Work

Planned future work includes a mobile-first progressive web app with offline support and push notifications; a two-way voice mode (continuous transcription in, speech out); broader OCR language coverage and per-language persona tuning; moving all audio paths to a fully self-hosted Whisper service; a marketplace where users share and rate workflow chains; optional per-user quotas and billing for a multi-tenant deployment; and a larger-scale performance and reliability evaluation.

REFERENCES

- [1] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Advances in Neural Information Processing Systems, 2020.
- [2] V. Karpukhin et al., “Dense Passage Retrieval for Open-Domain Question Answering,” Proc. EMNLP, 2020.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Proc. EMNLP-IJCNLP, 2019.
- [4] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” Proc. ICLR, 2023.
- [5] K. Schwaber and J. Sutherland, “The Scrum Guide,” 2020. [Online]. Available: <https://scrumguides.org>
- [6] Mistral AI, “Mistral models documentation.” [Online]. Available: <https://docs.mistral.ai>
- [7] HuggingFace, “Inference API documentation.” [Online]. Available: <https://huggingface.co/docs/api-inference>
- [8] n8n GmbH, “n8n documentation.” [Online]. Available: <https://docs.n8n.io>
- [9] PostgreSQL Global Development Group, “PostgreSQL 16 documentation.” [Online]. Available: <https://www.postgresql.org/docs/>
- [10] Docker Inc., “Docker documentation.” [Online]. Available: <https://docs.docker.com>
- [11] OpenJS Foundation, “Express 4.x API reference.” [Online]. Available: <https://expressjs.com>
- [12] AstroBot source repository. [Online]. Available: <https://github.com/tkonate788/astrobot>

CURRICULUM VITAE

Tidiane KONATÉ

PERSONAL INFORMATION

Full Name: Tidiane KONATÉ

Nationality: Malian (Mali)

Place and Date of Birth: May 24, 2004 in Bamako

E-mail: Tkonate788@gmail.com · 220201838@ostimteknik.edu.tr

EDUCATION

Degree	Institution	Graduation Year
High School	Lycée Planete Enfants (LPE), Mali — Scientific Baccalaureate	2021
Turkish Language Training	Yozgat Bozok University (YOBU), Turkey	2022
Undergraduate	OSTIM Technical University, Ankara — Computer Engineering	2026 (expected)

PROFESSIONAL EXPERIENCE

Institution	Role / Activities	Year
INEDIA — Lille, France (Remote)	Applied GenAI / LLMops Intern — designing and deploying AI-powered automation workflows with n8n, integrating LLM APIs into business processes, containerised service deployment with Docker, end-to-end GenAI pipelines.	2025 – present

AREA OF EXPERTISE

- Applied Generative AI & LLM APIs
- AI workflow automation & orchestration (n8n)
- DevOps / LLMops — Docker, webhooks, API integration
- Software development — C, C++, Python, C#
- Operating systems & networking

FOREIGN LANGUAGES

Language	Proficiency Level
French	Native
English	Advanced (C1)
Turkish	Intermediate (B1)
Bambara	Fluent

CERTIFICATES

OTHER RELEVANT QUALIFICATIONS

- Hands-on experience deploying containerised AI services and orchestration workflows on remote servers.

PUBLICATIONS

PRESENTATIONS

Sidi Mohamed SALL

PERSONAL INFORMATION

Full Name: Sidi Mohamed SALL

Nationality: Malian (Mali)

Place and Date of Birth: September 1, 2003 in Bamako

E-mail: Sidimohamedsall5@gmail.com · 210208992@ostimtechnik.edu.tr

EDUCATION

Degree	Institution	Graduation Year
High School	Lycée Complexe Scolaire Kanouté Aïssé, Bamako, Mali	2020
English Language Proficiency	OSTIM Technical University, Ankara	2021
Undergraduate	OSTIM Technical University, Ankara — Software Engineering	2026 (expected)

PROFESSIONAL EXPERIENCE

—

AREA OF EXPERTISE

- Product vision, user experience and interface design
- Front-end development with HTML, CSS and JavaScript
- Software engineering — analysis, design and documentation
- Team collaboration and agile project work

FOREIGN LANGUAGES

Language	Proficiency Level
French	Native
English	Advanced (C1)
Turkish	Intermediate (B1)
Bambara	Fluent

CERTIFICATES

—

OTHER RELEVANT QUALIFICATIONS

—

PUBLICATIONS

—

Ahmed ESSALEM

PERSONAL INFORMATION

Full Name: Ahmed ESSALEM

Nationality: Mauritanian (Mauritania)

Place and Date of Birth: Mauritania

E-mail: 220201992@ostimteknik.edu.tr

EDUCATION

Degree	Institution	Graduation Year
Undergraduate	OSTIM Technical University, Ankara — Computer Engineering	2026 (expected)

PROFESSIONAL EXPERIENCE

Institution	Role / Activities	Year
OSTIM Technical University	C#: Advanced Object-Oriented Programming (OOP), multi-module application development, scripting, automation, API integration and back-end development.	2026
Datacamp	Python: Advanced Object-Oriented Programming (OOP), multi-module application development, scripting, automation, API integration and back-end development.	2024
Smart SA Company	Database Management — SQL databases, data modelling, CRUD operations and database integration in applications.	2025
Smart SA Company	Web Technologies — frontend and backend development, responsive web applications and API-based systems.	2024

AREA OF EXPERTISE

- Computer Engineering: systems architecture, software development life cycle (SDLC), problem solving, debugging and application optimisation.
- Python Programming: advanced OOP, multi-module applications, scripting, automation, API integration and back-end development.
- C# Programming: advanced OOP, multi-module applications and back-end development.
- Software Engineering: application design, clean code practices, version control with Git/GitHub, testing and documentation.
- Database Management: SQL databases, data modelling and integration.
- Web Technologies: responsive web applications and API-based systems.

- Team Collaboration & Problem Solving: agile teamwork, technical communication and analytical thinking.

FOREIGN LANGUAGES

Language	Proficiency Level
Arabic	Fluent
French	Fluent
Turkish	Advanced (B2)
English	Advanced (C1)

CERTIFICATES

—

OTHER RELEVANT QUALIFICATIONS

—

PUBLICATIONS

—

PRESENTATIONS

—

Saad Ibrahim HOUSSEIN

PERSONAL INFORMATION

Full Name: Saad Ibrahim HOUSSEIN

Nationality: Djiboutian (Djibouti)

Place and Date of Birth: Djibouti

E-mail: saadgodin2@gmail.com · 210201983@ostimteknik.edu.tr

EDUCATION

Degree	Institution	Graduation Year
Undergraduate	OSTIM Technical University, Ankara — Software Engineering	2026 (expected)

PROFESSIONAL EXPERIENCE

Institution	Role / Activities	Year
Datacomp	Flutter Development — cross-platform mobile and web application development using Dart, responsive UI/UX design, state management, Firebase integration and REST API connectivity.	2024
Datacomp	Database Management — SQL databases, data modelling, CRUD operations and database integration in applications.	2025
Datacomp	Web Technologies — frontend and backend development, responsive web applications and API-based systems.	2024

AREA OF EXPERTISE

- Computer Engineering: systems architecture, software development life cycle (SDLC), problem solving, debugging and application optimisation.
- Python Programming: advanced OOP, multi-module applications, scripting, automation, API integration and back-end development.
- Flutter Development: cross-platform mobile and web apps with Dart, responsive UI/UX, state management, Firebase and REST APIs.
- Software Engineering: application design, clean code practices, version control with Git/GitHub, testing and documentation.
- Database Management: SQL databases, data modelling and integration.
- Web Technologies: responsive web applications and API-based systems.
- Team Collaboration & Problem Solving: agile teamwork, technical communication and analytical thinking.

FOREIGN LANGUAGES

Language	Proficiency Level
Somali	Fluent
Arabic	Fluent
French	Fluent
Turkish	Advanced (C1)
English	Advanced (C1)

CERTIFICATES

—

OTHER RELEVANT QUALIFICATIONS

—

PUBLICATIONS

—

PRESENTATIONS

—

13. Appendices

13.1. Appendix A: Source Code

Representative excerpts of the source code follow; the complete codebase is available at <https://github.com/tkonate788/astrobot>.

A.1 *JWT authentication middleware (backend/middleware/auth.js)*

```
const jwt = require('jsonwebtoken');

const authenticateToken = (req, res, next) => {
  const authHeader = req.headers['authorization'];

  if (!authHeader) {
    return res.status(401).json({
      success: false,
      error: 'Access denied. No authorization header provided.',
    });
  }

  const parts = authHeader.split(' ');
  if (parts.length !== 2 || parts[0] !== 'Bearer') {
    return res.status(401).json({
      success: false,
      error: 'Invalid authorization format. Use: Bearer <token>',
    });
  }

  const token = parts[1];

  if (!token) {
    return res.status(401).json({
      success: false,
      error: 'Access denied. Token missing.',
    });
  }

  try {
    const secret = process.env.JWT_SECRET || 'change_me_in_env';
    const decoded = jwt.verify(token, secret);
    req.user = decoded;
    next();
  } catch (err) {
    if (err.name === 'TokenExpiredError') {
      return res.status(401).json({
        success: false,
        error: 'Token expired. Please log in again.',
      });
    }
  }
  return res.status(403).json({
```

A.2 *Database pool and idempotent migrations (backend/db/index.js)*

```
const { Pool } = require('pg');
require('dotenv').config();

const pool = new Pool({
  host: process.env.DB_HOST || 'postgres_container',
  port: parseInt(process.env.DB_PORT) || 5432,
  database: process.env.DB_NAME || 'mydatabase',
```

```

user: process.env.DB_USER || 'tidiane',
password: process.env.DB_PASSWORD || 'changeme',
max: 20,
idleTimeoutMillis: 30000,
connectionTimeoutMillis: 5000,
});

pool.on('error', (err) => {
  console.error('[DB] Unexpected pool error:', err.message);
});

const initDB = async () => {
  const client = await pool.connect();
  try {
    console.log('[DB] Connected to PostgreSQL successfully.');
```

```

    await client.query(`
      CREATE TABLE IF NOT EXISTS users (
        id SERIAL PRIMARY KEY,
        name VARCHAR(100) NOT NULL,
        surname VARCHAR(100) NOT NULL,
        email VARCHAR(255) UNIQUE NOT NULL,
        password_hash TEXT NOT NULL,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);

    await client.query(`
      CREATE TABLE IF NOT EXISTS conversations (
        id SERIAL PRIMARY KEY,
        user_id INTEGER REFERENCES users(id) ON DELETE CASCADE,
        message TEXT NOT NULL,
        response TEXT,
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
      );
    `);

    // — Idempotent migrations: add session + media columns if missing —
    const CONV_COLUMN_ADDS = [
      ['session_id', 'VARCHAR(255)'],
      ['image_url', 'TEXT'],           // data URL of bot-generated image
      ['pdf_url', 'TEXT'],             // data URL of bot-generated PDF
      ['pdf_filename', 'VARCHAR(255)'],
      ['attachment', 'TEXT'],          // JSON: { name, mime, size, dataUrl }
    ];
    for (const [col, type] of CONV_COLUMN_ADDS) {
      await client.query(
        `ALTER TABLE conversations ADD COLUMN IF NOT EXISTS ${col} ${type};`
      );
    }

    // Add user avatar column (data URL of cropped profile photo)
    await client.query(`ALTER TABLE users ADD COLUMN IF NOT EXISTS avatar TEXT;`);
  }
}

```

A.3 Express entry point and security middleware (backend/server.js)

```

require('dotenv').config();

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');
const path = require('path');
const { initDB } = require('./db/index');
```

```

const authRoutes = require('./routes/auth');
const chatRoutes = require('./routes/chat');
const featuresModule = require('./routes/features');
const featuresRoutes = featuresModule.router;

const app = express();
const PORT = process.env.PORT || 3000;

// — Security Middleware —————
app.use(
  helmet({
    contentSecurityPolicy: false,
    crossOriginEmbedderPolicy: false,
  })
);

// — CORS —————
app.use(
  cors({
    origin: true,
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE', 'OPTIONS'],
    allowedHeaders: ['Content-Type', 'Authorization'],
  })
);

// — Rate Limiting —————
const globalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 200,
  standardHeaders: true,
  legacyHeaders: false,
  message: {
    success: false,
    error: 'Too many requests. Please try again later.',
  },
});

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 20,
  standardHeaders: true,
  legacyHeaders: false,
  message: {
    success: false,
    error: 'Too many authentication attempts. Please try again later.',
  },
});

```

13.2. Appendix B: Sprint Backlog

Figure 13.1 shows the project timeline; the sprint goals and the increments delivered in each are listed in Table 4.2 in Section 4. Each sprint maintained a backlog of user stories with priority and acceptance criteria; items not finished in a sprint were re-ranked for the next one, and the advisor's review feedback was added to the backlog as new items.

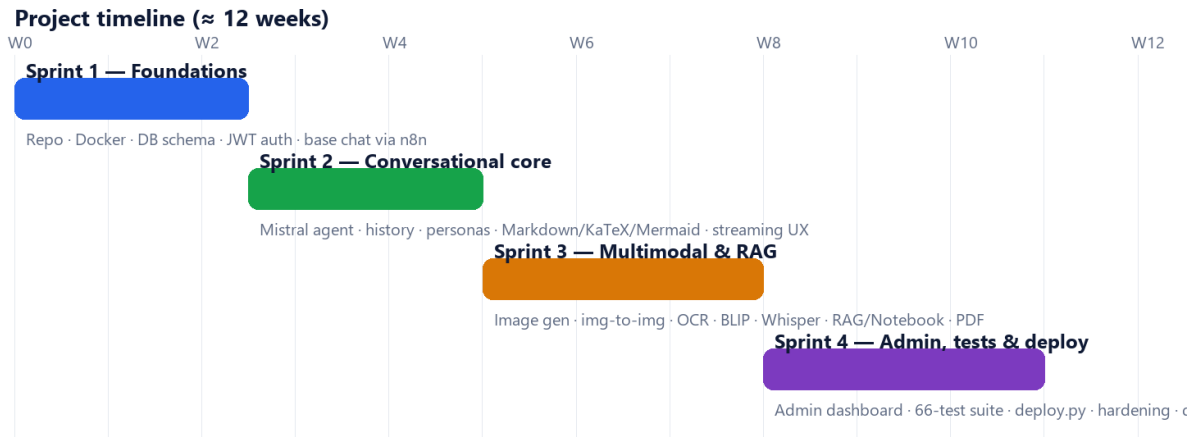


Figure 13.1: Sprint timeline of the project.

Sprint	Selected user stories (abridged)	Status
1	Set up repo & Docker Compose; create DB schema; implement register/login with JWT; minimal chat via n8n	Done
2	Mistral AI Agent; conversation history (15 msg); 7 personas; Markdown/KaTeX/highlight.js/Mermaid; streaming-style reveal	Done
3	Image generation & img-to-img; OCR + BLIP; Whisper transcription; RAG (chunk/embed/retrieve) & notebook mode; PDF report generation; workflow chains; tags & exports	Done
4	Admin dashboard (stats, users, audit); 66-case integration test suite; deploy.py; security headers & rate limits; documentation	Done

13.3. Appendix C: User Guide

Quick-start instructions for the deployed application:

- **Public application** — open <http://76.13.62.195:3000>, register an account (or log in), and start a conversation from the central chat bar. Use the persona selector to change the assistant's tone, the attachment button to add images (analysed by OCR/BLIP) or a PDF (then choose notebook or RAG mode), and the microphone button to dictate a message. Ask for an image ("generate an image of ...") or a report ("make a PDF report of ...") and the result appears inline. Tag, search, and export sessions from the sidebar.
- **Administrator** — open <http://76.13.62.195:7040> and log in with an admin account. The dashboard shows KPIs and activity charts; the Users page lets you search, suspend, reactivate, reset the password of, delete, or create accounts; opening a user shows their conversation history grouped by session.

- Deployment — from the repository, copy `docker/.env.example` to `docker/.env` and fill in the secrets, then run `docker compose up -d --build` in the `docker/` folder; or run `python deploy.py` to deploy to the configured server. Run `python test_all.py` to execute the integration suite against the deployment.
- Troubleshooting — if the public application is unreachable, check that the shared PostgreSQL container is running (`docker ps`); the AstroBot services restart automatically on a host reboot but the database must also be up for them to start serving.